

PROMPT KIMI K3 — NOTIFICACOES PUSH + TELEGRAM

Objetivo: Alertas de dividas e compromissos no celular sem app nativo

CONTEXTO (ja existe, NAO recriar)

Repo: heitorfragavd-prog/wallet-App | Stack: React 18 + TS + Vite + Supabase + shadcn/ui Padrao hooks: useQuery + useMutation + invalidateQueries + workspace_id + toast Padrao Edge Functions: supabase/functions/nome/index.ts com corsHeaders + createClient

PARTE 1: NOTIFICACOES PUSH DO NAVEGADOR (Web Push API)

1.1 Banco de Dados

```
-- Tabela para armazenar subscriptions de push
CREATE TABLE IF NOT EXISTS push_subscriptions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,
  endpoint text NOT NULL,
  p256dh text NOT NULL,
  auth text NOT NULL,
  created_at timestamptz DEFAULT now()
);

ALTER TABLE push_subscriptions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users manage own push subscriptions" ON push_subscriptions FOR ALL USING (auth.uid() =
  ⇨ user_id);
CREATE INDEX idx_push_user ON push_subscriptions(user_id);

-- Tabela para log de notificacoes enviadas
CREATE TABLE IF NOT EXISTS notificacoes_log (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  tipo text NOT NULL CHECK (tipo IN ('push', 'telegram')),
  titulo text NOT NULL,
  mensagem text NOT NULL,
  enviado boolean DEFAULT false,
  erro text,
  created_at timestamptz DEFAULT now()
);

ALTER TABLE notificacoes_log ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "Users view own notifications" ON notificacoes_log FOR ALL USING (auth.uid() = user_id);
```

1.2 Service Worker (public/sw.js)

Criar arquivo public/sw.js:

- Escutar evento push
- Mostrar notificacao com titulo, mensagem, icone
- Ao clicar na notificacao, abrir o app na rota correta
- Escutar evento notificationclick para abrir /dividas ou /agenda

1.3 Hook: useNotificacoesPush.ts

Criar em src/hooks/useNotificacoesPush.ts:

- registerPush(): pede permissao ao navegador, gera subscription via navigator.serviceWorker.ready + pushManager.subscribe, salva no Supabase
- unregisterPush(): remove subscription do Supabase e do navegador
- isSupported(): verifica se navegador suporta push
- getPermissionStatus(): retorna 'granted' | 'denied' | 'default'

1.4 Edge Function: supabase/functions/enviar-push/index.ts

Criar Edge Function:

- Recebe: user_id, titulo, mensagem, url (opcional)
- Busca todas as push_subscriptions do usuario
- Para cada subscription, envia notificacao via web-push
- Usar VAPID keys (ler de env vars: VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY)
- Registrar sucesso/falha na tabela notificacoes_log

1.5 Edge Function: supabase/functions/cron-notificacoes/index.ts (atualizar)

Modificar a Edge Function existente (ou criar nova):

- Roda diariamente (cron job Supabase)
- Busca dividas com vencimento em D+1, D+3
- Busca despesas do dia
- Para cada usuario com divida/proxima, chama enviar-push
- Para cada usuario com Telegram configurado, chama enviar-telegram

1.6 Modificar App.tsx

No useEffect de inicializacao:

```
useEffect(() => {
  if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/sw.js').catch(console.error);
  }
}, []);
```

1.7 Pagina: src/pages/ConfiguracoesNotificacoes.tsx

Criar pagina simples:

- Toggle: “Ativar notificacoes no navegador”
- Botao: “Testar notificacao” (envia push de teste)
- Status da permissao (permitido/bloqueado/pendente)
- Instrucoes para habilitar no celular

PARTE 2: BOT DO TELEGRAM

2.1 Banco de Dados

```
-- Tabela para vincular usuario ao Telegram
CREATE TABLE IF NOT EXISTS usuarios_telegram (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL UNIQUE,
  telegram_chat_id text NOT NULL,
  telegram_username text,
  ativo boolean DEFAULT true,
  created_at timestamptz DEFAULT now()
);

ALTER TABLE usuarios_telegram ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users manage own telegram" ON usuarios_telegram FOR ALL USING (auth.uid() = user_id);
```

2.2 Edge Function: supabase/functions/telegram-webhook/index.ts

Criar Edge Function:

- Recebe webhooks do Telegram (quando usuario manda mensagem pro bot)
- Comando /start: gera token unico, responde: “Conecte sua conta Wallet: [link com token]”
- Comando /dividas: lista dividas pendentes do usuario
- Comando /saldo: mostra saldo consolidado
- Comando /ajuda: lista comandos disponiveis

2.3 Edge Function: supabase/functions/enviar-telegram/index.ts

Criar Edge Function:

- Recebe: user_id, mensagem
- Busca telegram_chat_id na tabela usuarios_telegram
- Envia mensagem via API do Telegram (<https://api.telegram.org/bot%7BTOKEN%7D/sendMessage>)
- Token do bot: ler de env var TELEGRAM_BOT_TOKEN
- Registrar na notificacoes_log

2.4 Hook: useTelegram.ts

Criar em src/domains/finance/hooks/useTelegram.ts:

- vincularConta(token): chama RPC ou endpoint para vincular user_id ao telegram_chat_id
- desvincularConta(): remove registro da tabela
- isVinculado(): verifica se usuario tem telegram_chat_id

2.5 Pagina: src/pages/ConfiguracoesTelegram.tsx

Criar pagina:

- QR Code ou link para adicionar bot (@wallet_app_bot — exemplo)
- Instrucoes: “Abra o Telegram, busque @wallet_app_bot, mande /start”
- Status: Vinculado / Nao vinculado
- Botao “Desvincular”
- Botao “Testar mensagem”

2.6 Modificar ConfiguracoesNotificacoes.tsx

Juntar Push + Telegram em uma unica pagina:

```
Configuracoes de Notificacoes
├─ Notificacoes no Navegador [Toggle] [Testar]
├─ Bot do Telegram [Status] [Vincular] [Testar]
└─ Logs de notificacoes (ultimas 10 da tabela notificacoes_log)
```

PARTE 3: ROTAS E MENU

App.tsx

Adicionar:

```
const ConfiguracoesNotificacoes = lazyWithRetry(() => import("./pages/ConfiguracoesNotificacoes"));
<Route path="/configuracoes/notificacoes" element={<ProtectedRoute><ConfiguracoesNotificacoes /></ProtectedRoute>} />
```

Menu lateral

Adicionar item:

```
{ title: "Notificacoes", href: "/configuracoes/notificacoes", icon: Bell }
```

Importar Bell do lucide-react.

PARTE 4: VARIÁVEIS DE AMBIENTE (Supabase Secrets)

Adicionar no Supabase (Settings > Secrets):

```
VAPID_PUBLIC_KEY=BLxxxxxxxx...  
VAPID_PRIVATE_KEY=xxxxxxxx...  
TELEGRAM_BOT_TOKEN=123456789:ABCdef...
```

Gerar VAPID keys:

```
npm run web-push generate-vapid-keys
```

Criar bot no Telegram:

1. Abra Telegram, busque @BotFather
2. Mande /newbot
3. Escolha nome e username
4. Copie o token

PARTE 5: CHECKLIST

- public/sw.js (service worker)
- src/hooks/useNotificacoesPush.ts
- src/domains/finance/hooks/useTelegram.ts
- src/pages/ConfiguracoesNotificacoes.tsx (Push + Telegram juntos)
- supabase/functions/enviar-push/index.ts
- supabase/functions/enviar-telegram/index.ts
- supabase/functions/telegram-webhook/index.ts
- supabase/functions/cron-notificacoes/index.ts (atualizar)
- Migration SQL (push_subscriptions, notificacoes_log, usuarios_telegram)
- App.tsx: registrar service worker + nova rota
- Menu: item Notificacoes
- TypeScript 0 erros
- Build OK

REGRAS:

- Preservar TODOS os arquivos existentes
- Sempre workspace_id

- Sempre React Query
- Sempre toast
- Sempre RLS